1. **How to install Angular**
   npm install -g @angular/cli
   ng new <project-name>

2. **Latest Angular version**
   Angular 22

3. **Tell me something about Angular**
   Angular architecture is a highly structured, **component-based framework** designed by Google for building single-page applications (SPAs).

4. **What is Angular and how is it different from AngularJS?**
   AngularJS, released in 2010, is the older, JavaScript-based version. while Angular, introduced in 2016, is a complete rewrite using TypeScript

5. **Explain Angular architecture.**
   – Angular follows a component-based architecture that organizes an application into reusable building blocks.
   – This structured approach improves maintainability, scalability, and reusability. So making Angular suitable for both small and enterprise-level applications.
   – Below are the Core Architectural Building Blocks
   - **Components & Templates**
   - **Services & Dependency Injection (DI)**
   - **Directives:** modify the DOM or extend HTML element behaviors.
     - *Structural Directives* manipulate element layout (e.g., `@if`, `@for`).
     - *Attribute Directives* alter dynamic styling or element state (e.g., `ngClass`, `ngStyle`).
   - **Pipes:** Transform data visually within templates without modifying underlying data structures (e.g., `{{ date | date:'short' }}`).
   - **Routing**

- **Modules (NgModules):** Legacy container blocks (`@NgModule`) used to group related capabilities (components, directives, imports). While optional in modern Angular due to Standalone Components, they remain common in legacy codebases.

6. **What are components, directives, pipes, and services?**
   Attribute Directives

```
import { NgClass, NgStyle } from '@angular/common';
import { Component } from '@angular/core';
import { NgClass, NgStyle } from '@angular/common';

@Component({
selector: 'app-root',
standalone: true,
imports: [NgClass, NgStyle],
template: `
      <div [ngClass]="{'highlight': isHighlighted, 'italic':
      isItalic}">
      Dynamic classes applied
      </div>
      <div [ngStyle]="{'color': textColor, 'font-size': fontSize +
      'px'}">
      Dynamic styles applied
      </div>
      `
})
export class AppComponent {
isHighlighted = true;
isItalic = false;
textColor = 'blue';
fontSize = 18;
}
```

Structural Directives

```
Import { CommonModule } from '@angular/common';

@Component({
imports: [CommonModule],
template: `
      <p *ngIf="items.length > 0">We have {{ items.length }}
            items</p>
      <ul>
      <li *ngFor="let item of items">{{ item }}</li>
      </ul>
`
```

```
})
```

7. **What is a standalone component?**
   Standalone components allow each component to **declare its own dependencies** directly, eliminating the need for a central NgModule to manage imports, declarations, and providers. This approach makes applications **simpler to reason about, easier to maintain, and faster to develop**

8. **What is an NgModule? Is it still required?**
   NgModule is no longer required for new Angular applications.As of the current Angular version, **Angular 22**, the recommended approach for new code is **Standalone Components**
   An `NgModule` is a class decorated with `@NgModule` that historically grouped related Angular components, directives, pipes, and providers.

   app.module.ts

```
@NgModule({
  declarations: [
    App,
    First
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [
    provideBrowserGlobalErrorListeners(),
  ],
  bootstrap: [App]
})
export class AppModule { }
```

24. Explain Angular component lifecycle hooks.

Hook	Simple meaning	When it runs
`ngOnChanges()`	Input data changed	When a value received through `@Input()` changes
`ngOnInit()`	Component started	Once, after Angular sets the initial inputs
`ngDoCheck()`	Check for changes	During Angular's change-detection checks
`ngAfterContentInit()`	Content inserted	Once, after projected content (`ng-content`) is initialized
`ngAfterContentChecked()`	Projected content checked	After Angular checks projected content
`ngAfterViewInit()`	View is ready	Once, after the component's template/view is initialized
`ngAfterViewChecked()`	View checked	After Angular checks the component's view
`ngOnDestroy()`	Component is going away	Just before Angular destroys the component

25. Difference between `constructor()` and `ngOnInit()`.

Quick Comparison

Feature	Constructor	ngOnInit
Origin	Native TypeScript / ES6 Class feature	Angular-specific lifecycle hook
Execution Order	Runs **first**, when the class is instantiated	Runs **second**, after the constructor and `ngOnChanges`
Triggered By	The JavaScript engine	The Angular framework
Access to `@Input()`	❌ No (inputs are not yet bound)	Yes (inputs are fully accessible)
Primary Purpose	Dependency Injection (DI) & basic class field setup	Complex initialization (API calls, DOM logic, event listeners)

26. What is `ngOnChanges()` and when is it triggered?

`ngOnChanges()` is an Angular lifecycle hook method that runs when input properties (`@Input()`) on a component change.

27. Difference between `ngOnInit()` and `ngAfterViewInit()`.

Feature	`ngOnInit()`	`ngAfterViewInit()`
Execution Timing	Runs early in the lifecycle, immediately after the first `ngOnChanges()`.	Runs later, after the entire view and child components are fully rendered.

What is `ngOnDestroy()` used for?

In Angular, `ngOnDestroy()` **is a lifecycle hook used to perform manual cleanup just before Angular destroys a component, directive, pipe, or service.**

```
ngOnDestroy() {
    // Clean up RxJS subscription
    this.mySubscription.unsubscribe();
    // Clean up native timer
    clearInterval(this.intervalId);
}
```

28. **How does Angular detect changes?**
    Angular detects changes through a process called change detection and to find out change detection, Angular uses one library called Zone.js.

29. **What is Zone.js?**
    `zone.js` is a core library used by Angular to automatically trigger change detection by intercepting asynchronous operations. It functions like an invisible background worker that keeps track of when your application's state might have changed so that Angular knows exactly when to update the HTML view

30. **What is Angular's dependency injection system?**
    Dependency Injection is a core design pattern which creates a single and unique dependent object and shares it through the application. Instead of forcing a component or service to manually instantiate its own dependencies using `new ClassName()`, Angular manages their lifecycle and hands them over on demand
31. **What is the difference between `providedIn: 'root'` and component-level providers?**
    `providedIn: 'root' =>` create global instance of service instead of creating using New ClassName, So this will prevent new object creation everytime.
32. **What is hierarchical dependency injection?**
    Normally Angular instantiates a single object using ProvidedIn: root, but in case If we want to create a separate instance, then you can define it in the component itself, inside the Providers array.
33. **How does Angular resolve a dependency?**
    When a component requests a dependency (either by constructor or inject() method) then angular tries to find out the hierarchical lookup process. And that's how it resolves dependency. Child -> parent -> parent -> app component.
34. **What is `inject()` and how is it different from constructor injection?**
    It allows you to add a dependency using a standard function call instead of declaring it as a parameter in a class constructor. For the constructor injection, you need to manually pass every single dependency from the child constructor up to the parent constructor and this will create messy and repetitive code.

35. **What are Angular decorators?**

decorators are special TypeScript functions prefixed with an `@` symbol that add metadata to your classes, methods, properties, or parameters. They tell the Angular compiler how to process and behave with your code without changing its logic

Below are the different types of decorators:

1. Class Decorators

These are applied directly above a class to configure its identity and behavior.

- `@Component`: Configures a class as a UI component by defining its template, selector, and styles.
- `@Injectable`: Marks a class as a service that can be managed by Angular's Dependency Injection system.
- `@Directive` & `@Pipe`: Mark classes as directives or data transformers.

2. Property Decorators

These are applied to properties (fields) inside a class to manage data flows.

- `@Input()`: Allows a parent component to pass data down to this property.
- `@Output()`: Emits custom events up to a parent component.

3. Method Decorators

These are placed above methods to handle specific events or intercept execution.

- `@HostListener()`: Listens to events triggered on the host element (like clicks or window resizing) and executes the method.

Data Bindings

36. **What are the different types of data binding in Angular?**

– Oneway Binding - Component to HTML file

– Interpolation - Welcome, {{ username }}

– Property binding - <button [disabled]="isButtonDisabled">Submit</button>

– Event binding - <button (click)="saveData()">Save</button>

– Attribute Binding - <td [attr.colspan]="columnSpan"></td>

– Class Binding - <div [class.active]="isActive">Content
– Style Binding - <button [style.color]="isError ? 'red' : 'blue'">Click</button>

– Two-way binding

```
<input type="text" [(ngModel)]="fruiteName"
id="textbox" name="textbox"/>
<p><button (click)="changeName('Mango')">click me to
change Fruite</button></p>

fruiteName = "Banana";
changeName(newName: string){
    this.fruiteName = newName;
}
```

40. Difference between: [value]="name" and {{ name }}

```
isDisabled = false;

<button [disabled]="isDisabled">Save using property
Binding</button> // This will be enable

<button disabled="{{isDisabled}}">Save using
Interpolation</button> // This will be disable as it will
interpret as string “false”
```

41. How does `[(ngModel)]` work internally?

Angular splits the `[(ngModel)]` "banana-in-a-box" syntax into a combination of a property binding and an event binding.
`[ngModel]="property"` and
`(ngModelChange)="property = $event".`

42. **What is property binding vs attribute binding?**
    – property binding updates the Document Object Model (DOM) property.
    `<button [disabled]="isFormInvalid">Submit</button>`
    – Attribute binding updates the HTML attribute
    `<td [attr.colspan]="numberOfColumns">Data Cell</td>`

43. **Difference between: [attr.disabled] and [disabled]**
    [disabled] - target DOM element
    [attr.disabled] - target HTML attribute
    <button [disabled]="isDisabled"> Property Binding </button>
    <button [attr.disabled]="isDisabled"> Attribute Binding </button>
    <button [attr.disabled]="isDisabled ? '' : null">Attribute Binding</button>

44. **What is event bubbling?**
    When an event happens on a child element, the event can travel upward through its parent elements.
    Think of it like a bubble going from the inside → outside.

```
<div (click)="showMessage('DIV clicked')">
  <button
     (click)="showMessage('BUTTON clicked')">
     Click Me
  </button>

</div>

BUTTON clicked
DIV clicked

<button
(click)="$event.stopPropagation();showMessage('BUTTON
clicked')">
Click Me
</button>

BUTTON clicked
```

45. How can a child component communicate with its parent?

```
##Child Component
nameChange = new EventEmitter<string>();
public injectService = inject(AppService);

changeNameInParent(){
   console.log("Child component event");
   // Using @Output
   //this.nameChange.emit("Bhavesh");
   // Using Observables
   this.injectService.onNameChange("Bhavesh");
}

##Service
nameChange$ = new Subject<string>();
onNameChange(name: string){
    console.log("name change in service::", name);
    this.nameChange$.next(name);
}

##Parent Component
<child (nameChange) = "nameChangeEvent($event)"
[userId]="userId"></child>
nameChangeEvent(name: string){
  console.log("name in Parent Component::",name);
}
```

46. How can a parent communicate with its child?

47. Difference between `@Input()` and `@Output()`

@Input() is used to pass data from Child Component to Parent Component.
@Output() is used to pass data from Child to Parent component

48. **How do you share data between unrelated components?**
    I would generally use a shared injectable service as a communication bridge. Depending on the requirement, the service can expose an Observable using `Subject`/`BehaviorSubject`, or Signals in modern Angular. Components can publish or consume the shared state through that service.

Change Detection

49. **What is change detection?**
    Change Detection is Angular's mechanism to update DOM whenever any variable, object, or property updates in typescript code. Traditionally this is achieved using zone.js library. Now in new version, Angular has introduced Signals, which is very less dependent on Zone.js
50. **How does Angular's change detection work?**
    Change Detection is Angular's mechanism to update DOM whenever any variable, object, or property updates in typescript code. When any change is detected, Angular start traversing from Top(root component) to Bottom, and update the required DOM changes.
51. **What is the difference between:** ChangeDetectionStrategy.Default and ChangeDetectionStrategy.OnPush
    – `ChangeDetectionStrategy.Default` checks every component in the application tree on every event.
    `ChangeDetectionStrategy.OnPush` skips checking the component tree unless a specific trigger occurs, such as an `@Input` reference change.

```
##Parent Component HTML
<p><button id="cd"
(click)="changeDetectionDefault()">changeDetectionDefault
</button></p>
<p><button id="cdp"
(click)="changeDetectionOnPush()">changeDetectionOnPush</
button></p>
<p><app-change-detection-strategy [user] =
"user"></app-change-detection-strategy></p>
<p><app-change-detection-strategy-onpush [user] =
"user"></app-change-detection-strategy-onpush></p>
##Parent Component TS
changeDetectionDefault(){
```

```
   this.user.age = this.user.age + 1;
   console.log(" this.user.age::", this.user);
}
changeDetectionOnPush(){
    this.user = {...this.user,age:this.user.age+1};
    console.log("this.user.age::", this.user);
}
##Child Component
@Component({
  selector: 'app-change-detection-strategy-onpush',
  imports: [],
  templateUrl: './change-detection-strategy-onpush.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
  styleUrl: './change-detection-strategy-onpush.scss',
})
@Input user: {id: number, age: number}
ngOnChange(){

}
```

52. **Why doesn't changing an object property sometimes update an `OnPush` component?** For example, user.name = 'John';
    This will not change memory reference. So the onPush change detection method will not know about the change, so it will not update.
    {...this.user, 'name': "Bhavesh"}
53. What is `ChangeDetectorRef`?
    Angular is smart and updates every event automatically. But sometimes, If you want to take control and want to manually update the HTML than you can use `ChangeDetectorRef.` This is especially used when you are doing any heavy process or updating through a third party library. Than you can manually tell Angular to update

54. Difference between: markForCheck() & detectChanges().
    `markForCheck()` = "Angular, please check me later." This will be checked in the next detection cycle. Mostly used for OnPush CD strategy.

`detectChanges()` = **"Angular, check me right now."** This will force Angular to update all the parent and its child components.

55. When would you use `detach()`?
    Angular, please stop checking this component automatically until I tell you.It is mainly a **performance optimization** and used when you have too many records like 10,000 records.

```
    time = 0;

    constructor(private cdr: ChangeDetectorRef) {
      this.cdr.detach();

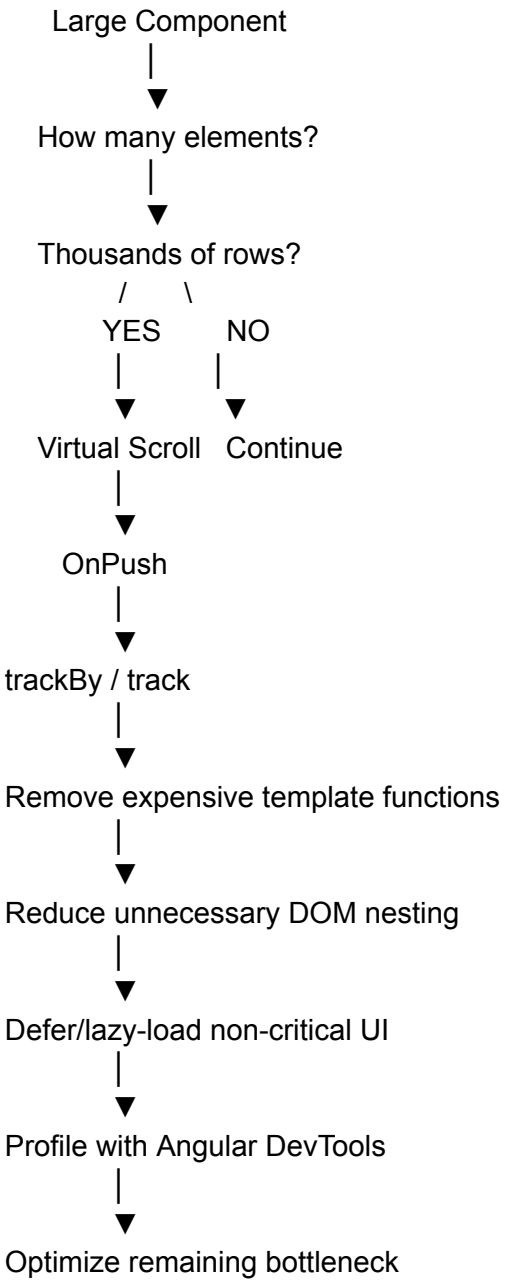
      setInterval(() => {
        this.time++;

        // Update UI manually
        this.cdr.detectChanges();

      }, 1000);
    }
// Here Angular doesn't continuously check the component.

  Every 1 second
       ↓
  time++
       ↓
  detectChanges()
       ↓
  UI updates
```

56. How would you optimize a component with thousands of DOM elements?
    First I'd determine whether all elements actually need to be rendered. For large lists I'd use virtual scrolling to keep the DOM small. I'd use `OnPush` change detection and `trackBy` or `track` so Angular can skip unnecessary checks and reuse existing DOM nodes. I'd avoid expensive template functions and getters, reduce unnecessary DOM nesting, and defer or lazy-load non-critical content. Finally, I'd profile the component with Angular DevTools and browser performance tools before and after the changes.



HTTP Client

57. How does Angular `HttpClient` work?
    Angular `HttpClient` is an injectable service class that handles HTTP communication between your front-end application and back-end servers. Angular uses the browser's fetch() API internally to request API calls. All standard `HttpClient` methods (such as `get`, `post`, `put`, `delete`) return an **RxJS Observable**.
58. Why does `HttpClient.get()` return an Observable?
    Angular implements **RxJS Observable** for the `HttpClient.get()` because Observable provides lots of benefits. It provides benefits like canceling API requests. There are lots of operators and functions that provide work around to manipulate data and data streams.
59. Are HTTP Observables cold or hot?
    HTTP Observables are **cold. In RxJS**, a cold observable is "lazy"—it does not produce data or execute any logic until an observer explicitly listens to it. The actual AJAX request is never fired until you call `.subscribe()` (or resolve it via the `async` pipe in a template).
60. How do you add HTTP headers?
    1. Object Literal (Simplest Approach)

```
// Inside your component or service
const http = inject(HttpClient);

http.get('/api/config', {
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer YOUR_TOKEN'
  }
}).subscribe(config => {
  // Handle response
});
```

    2. Using the `HttpHeaders` Class

```
import { HttpClient, HttpHeaders } from
'@angular/common/http';
```

```
const headers = new HttpHeaders({
  'Content-Type': 'application/json',
  'X-Debug-Level': 'verbose'
});

this.http.get('/api/config', { headers
}).subscribe();
```

61. **How do you handle HTTP errors globally?**
    We can add an HTTP interceptor to manage outgoing requests and incoming responses from the APIs. We can add errors globally and manage logger functionality. Also we should use HttpContext for managing metadata like whether the JWT token is set or not?
62. **What is an HTTP interceptor?**
    An HTTP Interceptor in Angular is a middleware mechanism that allows developers to inspect, transform, or handle outgoing HTTP requests and incoming HTTP responses globally before they reach the server or your application components
63. **How would you add JWT authentication using an interceptor?**
64. **How would you refresh an expired JWT token?**
65. **How do you prevent multiple refresh-token requests?**
66. **How do you retry failed HTTP requests?**
67. **Difference between HTTP `PUT` and `PATCH`.**
    PUT - For fewer field update
    PATCH - To replace all Form fields.
68. **How do you cancel an HTTP request? -** switchMap
69. **How do you cache API responses? -** shareReplay
70. **How do you globally show/hide a loading spinner?**
    – Using API Interceptor, Spinner Component, Signal services
71. **How would you handle `401`, `403`, `404`, and `500` responses?**
    401 Unauthorized : invalid credentials.
    403 Forbidden : Do not have access permission
    404 Not Found : page or resource does not exist
    500 Internal Server Error : backend server crashed or failed unexpectedly

Routing

72. **What is lazy loading?**
    Using lazy loading, you can load components on-demand, only when they are needed.

```
73. export const routes: Routes = [
```

```
  { path: '', pathMatch: 'full', redirectTo: 'login' },
  { path: 'login', component: Login, canActivate: [guestGuard] },
  // Normal Loading Component
  { path: 'about', component: About, canActivate: [authGuard] },
  // Lazy Loading Component
  {
    path: 'about',
    loadComponent: () => import('./about/about').then((m) => m.About),
  }
];
```

84. **Why should we lazy-load modules/routes?**
    To improve performance. Few component on home page like FAQ, policy page, Admin page are rarely used, so this type of page can be loaded on demand, to improve page loading time, and it will improve user experience.

85. **What are route guards?**
    canMatch - Conditional component routing
    canActivate - To decide where user can access that component
    canActivateChild - Instead of applying `canActivate` guard to each and every child **component.** You can add canActivateChild for the child component
    canDeactivate - This guard checks if a user is allowed to *leave* the current page Warning a user that they will lose unsaved text updates if they leave a form.
    resolve **-** Used to fetch data before components start rendering.

86. How do you pass route parameters?
    app.route.ts

```
export const routes: Routes = [
  // ':id' acts as a dynamic route parameter
  { path: 'product/:id', component: ProductDetailComponent }
];
```

HTML File

```
<a [routerLink]="['/product', product.id]">View
Product</a>

How To Access Route Parameter
this.route.paramMap.subscribe(params => {
  const id = params.get('id');
});
```

91. Route parameter vs query parameter.
    Route Parameter: Route parameters are compulsory parameters.
    Query Parameter: Optional

```
this.router.navigate(['/products'], { queryParams: {
category: 'shoes', page: 2 } });

// How to access Query Parameter
this.route.queryParamMap.subscribe(queryParams => {
  const category = queryParams.get('category');
});
```

92. How would you handle unsaved form changes when navigating away?
    ⇒ Using **canDeactivate** guard.

Forms ⭐⭐

93. Template-driven vs reactive forms.
    Template-driven - Traditional form with everything is managed using ngModel
    Reactive form - it is synchronous form and very helpful for larger form fields
94. Which one do you prefer for large applications and why?
    Reactive form

95. **What is `FormControl`?**
    Track value and validation of individual form input elements.

```
import { FormControl, ReactiveFormsModule, Validators }
from '@angular/forms';

imports: [RouterLink, ReactiveFormsModule],

name = new FormControl('Emilys', [Validators.required,
Validators.minLength(3)]);

//HTML
<input type="text" placeholder="Name" id="name"
[formControl]="name" />
```

96. What is `FormGroup`?
    Track value and validation for the group of individual form input elements.

```
<form [formGroup]="profileForm" (ngSubmit)="onSubmit()">
  <label for="first-name">First Name: </label>
  <input id="first-name" type="text"
formControlName="firstName">

  <label for="last-name">Last Name: </label>
  <input id="last-name" type="text"
formControlName="lastName">

  <button type="submit"
[disabled]="!profileForm.valid">Submit</button>
</form>
```

```
  // Create the FormGroup structure
   profileForm = new FormGroup({
    firstName: new FormControl('', Validators.required),
    lastName: new FormControl('')
  });

  onSubmit() {
    // Access the entire structured data object at once
    if (this.profileForm.valid) {
      console.log(this.profileForm.value);
      // Output: { firstName: 'John', lastName: 'Doe' }
    }
  }
```

97. What is `FormArray`?
    It is similar to FormGroup but it stores values in numerical arrays.
98. How do you create dynamic forms?
    Using reactive form
99. How do you create custom validators?

```
this.myForm = new FormGroup({
  website: new FormControl('', [Validators.required,
httpsValidator])
});

import { AbstractControl, ValidationErrors } from
'@angular/forms';

// Checks if a value starts with https://
export function httpsValidator(control: AbstractControl):
ValidationErrors | null {
  const value = control.value;
```

```
  if (!value) {
    return null;
  }
  const isValid = value.startsWith('https://');
  // Returns null if valid, or an error object if invalid
  return isValid ? null : { invalidHttps: true };
}
```

100. How do you validate two fields against each other? password & confirmPassword
Create a reusable validation helper function and use it inside reactive form validation.
101. How do you disable/enable controls?
this.form.get('controlName').disable()
this.form.get('controlName').enable()
102. Difference between: setValue() patchValue()
`setValue()` - require all fields to be updated from the formGroup.
`patchValue()` - can update individual value from multiple form Value
103. How do you detect form changes?

```
// 1. Listen to the entire form changes
this.profileForm.valueChanges
  .pipe(debounceTime(300)) // Optional: Wait 300ms for
user to stop typing
  .subscribe(newValue => {
    console.log('Form values changed:', newValue);
  });

// 2. Listen to a single control change
this.profileForm.get('firstName')?.valueChanges.subscribe
(name => {
  console.log('First name changed to:', name);
});
```

104. How do you reset a form?

```
this.profileForm.reset();
```

Performance Optimization ⭐⭐⭐

105. What is `OnPush`?
106. What is `trackBy`?

```
  items = [
    { id: 101, name: 'Laptop' },
    { id: 102, name: 'Mouse' },
    { id: 103, name: 'Keyboard' }
  ];

  // The trackBy function
  trackByItemId(index: number, item: any): number {
    return item.id; // Returns the unique ID of the item
  }

<ul>
  <li *ngFor="let item of items; trackBy: trackByItemId">
    {{ item.name }}
  </li>
</ul>
```

107. What is Angular CDK virtual scrolling?

Instead of forcing the browser to create thousands of Document Object Model (DOM) elements simultaneously, virtual scrolling **only renders the specific items currently visible inside the user's** page.

– To use virtual scrolling, first install **@angular/cdk** and import the ScrollingModule from @angular/cdk/scrolling into your component or module

```
 <cdk-virtual-scroll-viewport itemSize="50"
class="my-list-viewport">
  <div *cdkVirtualFor="let item of complexDataset; let i
= index" class="list-item">
    Item #{{ i }}: {{ item.title }}
  </div>
</cdk-virtual-scroll-viewport>
```

108. **How do you reduce unnecessary API calls?**
– Optimize User Inputs & Search (RxJS Filtering)

```
this.searchControl.valueChanges.pipe(
  // 1. Wait 300ms of silence before emitting
  debounceTime(300),
  // 2. Only emit if the current value is different from
the last
  distinctUntilChanged(),
  // 3. Optional: Block requests if text is too short
  filter(term => term.length >= 3),
  // 4. Cancel previous pending request, use latest
  switchMap(term => this.apiService.search(term))
).subscribe(results => this.results = results);
```

– **Prevent Duplicate Submission Bloat**

```
this.saveButtonClick$.pipe(
  exhaustMap(() =>
this.apiService.saveData(this.formData))
).subscribe(response => handleSuccess(response));
```

– **Cancel Active Requests on Route Changes**

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.apiService.getDashboardData().pipe(
    takeUntil(this.destroy$) // Automatically cancels
the HTTP stream when destroy$ emits
  ).subscribe(data => this.data = data);
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

– **Implement Response Caching**

```
@Injectable({ providedIn: 'root' })
export class StaticDataService {
  // The API is called exactly once; the last 1
response is cached and shared
  readonly countries$ =
this.http.get<Country[]>('/api/countries').pipe(
    shareReplay(1)
```

```
  );
}
```

– Centralized HTTP Interceptor Cache

109. **How do you implement API caching?**

Three primary ways to cache the API data

– HTTP Interceptor

– RxJS shareReply

– Browser's LocalStorage and sessionStorage

110. **How do you reduce bundle size?**

– When building Angular, keep option **–state json,** so it will list out total and individual file/package size.

– Use Lazy loading and load only required component

– import only required functions/operator

– Use lighter third party library, If you have to use

– Use Standalone component instead of ngModule–

111. **What is tree shaking?**

Think of your application as a physical tree: when you shake it, the dead leaves and branches (unused functions, components, or services) fall off, leaving only the solid branches (the code actually required to run the app) behind. Tree Shaking is a build time optimization technique that removes unused OR dead code from the final production bundle and thus it removes bundle size.

The most practical example of tree shaking in Angular is how services are registered using the `@Injectable` decorator.

```
import { Injectable } from '@angular/core';
@Injectable({
// Tells Angular to tree-shake this service if no one
imports it
  providedIn: 'root'
})
export class AnalyticsService {
  logEvent(event: string) {
    console.log(event);
  }
}
```

112. **What is code splitting?**

It is a build time optimization technique that breaks monolithic Javascript bundle into smaller chunk of code

113. **How do you identify performance bottlenecks?**

– Add Angular DevTool and check Profile tab - here you will see the component's maximum CD.

– Add a log for the ngDestroy to check the memory leakage.

– objects for the RxJS and event listeners must be destroyed.

114. **How would you optimize a page taking 5–10 seconds to load?**

– Lazy load routes

– Use Tree Shaking strategy to remove heavy library

– Enable production build *--configuration production*

– Content Delivery Network (CDN) to serve files faster from nearby locations

– Use OnPush change detection for the complex task

– Add trackBy functions: Use `trackBy` with `*ngFor` loops so Angular only updates changed items in a list instead of re-rendering everything

– Implement virtual scrolling: Use virtual scrolling for massive lists so the DOM only renders items visible on the screen.

115. **How would you optimize a table containing 10,000 records?**

– Server-side pagination

– Server-side filtering and sorting

– Debounced search

– Virtual scrolling - <cdk-virtual-scroll-viewport>

– Use trackBy: Always supply a `trackBy` function with `*ngFor` so Angular updates only modified or added rows instead of re-rendering the entire list

– OnPush change detection: Set components to `changeDetection: ChangeDetectionStrategy.OnPush` to reduce unnecessary checks across the component tree

– Avoid keeping complex function inside HTML template file

116. **How do you prevent memory leaks?**

To prevent memory leaks in Angular, you must properly manage active RxJS subscriptions, native event listeners, browser timers, and component references in singleton services. Start using Signals to prevent memory leakage and destroy

117. **How would you optimize images/assets?**

– To optimize images and assets in Angular, the most critical step is adopting the built-in `NgOptimizedImage` directive

– Prioritize Critical Images . add priority attribute to the image which required fast loading

118. What is lazy loading of images?

Import `NgOptimizedImage` into your component or module:



```
import { NgOptimizedImage } from '@angular/common';

@Component({
  imports: [NgOptimizedImage],
  // ...
})
```

– Use the `ngSrc` attribute instead of the standard `src`:

```
<!-- This will automatically lazy-load -->
<img ngSrc="assets/large-photo.jpg" width="800"
height="600" alt="Lazy Loaded Image">

<!-- For above-the-fold images, disable lazy loading
using priority -->
<img ngSrc="assets/hero-banner.jpg" width="1200"
height="400" priority alt="Hero Banner">
```

119. How do you optimize change detection?

Optimization Technique	Primary Benefit	Implementation Complexity
`OnPush` Strategy	Prevents deep rendering of unchanged sub-trees	Medium (Requires immutability)
Angular Signals	Achieves precise, localized DOM updates	Low (Recommended for new code)
`runOutsideAngular`	Stops background timers/events from firing UI updates	Low
Pure Pipes / `track`	Eliminates repetitive computations and DOM churn	Very Low